Project Report
on

Hostel Management System

Submitted By

Ayush Yadav (25IMC13004)

Subject Name: Technical Training – I
Subject Code: CAP-652

in partial fulfilment for the award of the degree of
MCA

IN
Artificial Intelligence and Machine Learning
University Institute of Computing
Department of Carrier Planning and Development

Academic Year: 2025–26

Under the Guidance of:
Mr. Shalabh Bhatiya
(E18471)

ACKNOWLEDGEMENT

We would like to express our sincere gratitude and appreciation to all those who have helped and guided us throughout the completion of this project on **Hostel Management System.**

First and foremost, we are deeply grateful to our faculty mentor for their constant encouragement, invaluable feedback, and academic guidance. Their expertise in database management systems and relational database design helped us navigate through the complexities of this project with confidence.

We would also like to acknowledge our fellow classmates whose constructive suggestions and collaborative spirit enriched our understanding of database normalization, trigger-based automation, and SQL query optimization.

Special thanks to our families for their unwavering moral support and encouragement throughout the duration of this academic project.

This project has been an invaluable learning experience, enhancing our practical understanding of PostgreSQL, schema design, stored procedures, views, and complex query formulation in a real-world context.

– Project Team

CERTIFICATE

This is to certify that the project entitled **"HOSTEL MANAGEMENT SYSTEM"** has been successfully completed by the following students as part of their academic curriculum for the academic session 2025-2026 for Second Semester. The work submitted is original and has not been submitted elsewhere for any degree or certification.

Students:

Priyanka Chandwani	MCA – AI&ML	25MCI10122
Sahil Hans	MCA – AI&ML	25MCI10088
Shubham Agarwal	MCA – AI&ML	25MCI10091
Ayush Yadav	MCA-AI&ML	25IMC13004

Faculty Guide

Signature: _____

Date: _____

ABSTRACT

The Hostel Management System is a relational database-driven application designed to digitize and streamline the administration of student hostel operations. Traditional hostel management involves significant paperwork, manual record-keeping, and inefficient communication, which often leads to allocation errors, delayed complaint resolution, and poor resource utilization.

This project presents a comprehensive PostgreSQL-based database schema that models four core entities — students, rooms, allocations, and complaints — each represented through normalized relational tables. The system leverages referential integrity through foreign key constraints, automated room occupancy tracking through database triggers, and flexible data retrieval through SQL views and stored functions.

Key features of the system include: automatic room occupancy updates on each student allocation via a AFTER INSERT trigger, a real-time view to display available rooms, a PL/pgSQL function to retrieve room details of any student, and constraint enforcement to prevent overbooking beyond room capacity. The project also demonstrates a comprehensive suite of SQL queries covering data retrieval, update operations, deletion, aggregation, and multi-table joins.

The database supports sixteen students across ten rooms with varying capacities, tracks thirteen complaint records with priority statuses, and provides query outputs for administrative decision-making. The system proves efficient in managing room allocations, tracking pending complaints, identifying unallocated students, and generating room-wise occupancy reports.

This project serves as a practical demonstration of core database management principles including entity-relationship modeling, normalization up to Third Normal Form (3NF), procedural SQL, and query optimization — making it a scalable foundation for a fully deployed hostel management application.

TABLE OF CONTENT

Sr. No.	Title	Page No.
1	Introduction	6
2	Objectives	7
3	System Requirements	7
4	Entity-Relationship Description	8
5	Table Structures & Creation	9
6	Constraints & Trigger & Function	10
7	Data Insertion (INSERT Queries)	11
8	SELECT Queries with Outputs	14
9	UPDATE Queries with Outputs	16
10	DELETE & Other Queries with Outputs	17
11	Conclusion	18
12	References	19

1. INTRODUCTION

A hostel management system is an essential administrative tool for educational institutions that house students in campus residences. With growing student populations and complex accommodation requirements, managing room assignments, tracking occupancy, and resolving student complaints manually has become increasingly impractical. The need for an efficient, automated, and reliable database-driven solution is paramount.

This project, titled Hostel Management System, is designed using PostgreSQL — one of the most powerful open-source relational database management systems. The system models the real-world scenario of a university hostel where multiple students are enrolled across various courses and years, rooms have varying capacities, and a formal complaint mechanism exists for reporting hostel-related issues.

The database schema is built around four primary tables: students (storing personal academic information), rooms (managing room inventory and occupancy), allocation (recording room assignments per student), and complaints (logging student grievances with tracking status). These tables are interlinked through foreign key relationships ensuring data integrity and consistency.

The project goes beyond simple table creation by incorporating advanced database features such as:

- Triggers — for automatic occupancy count updates on room allocation.
- Stored Functions — to fetch room information for any given student.
- Views — to dynamically display only rooms with available capacity.
- Check Constraints — to prevent overbooking beyond room capacity.
-

The SQL implementation covers all essential database operations including SELECT with JOINs and aggregation, UPDATE for record modification, DELETE for data cleanup, and STRING_AGG for consolidated room-wise student reporting. This report documents all SQL code, explains its purpose, and presents screenshots of execution outputs for each query section.

2. OBJECTIVES AND SYSTEM REQUIREMENTS

2.1 Objectives

The primary objectives of this Hostel Management System database project are:

- To design a relational database schema for managing hostel operations.
- To implement automated room occupancy tracking using database triggers.
- To provide a real-time view of available rooms for administrative use.
- To manage student room allocations with referential integrity and constraint enforcement.
- To implement a complaint management system with status tracking.
- To demonstrate various SQL operations including SELECT, INSERT, UPDATE, DELETE, and aggregation queries.
- To create stored PL/pgSQL functions for on-demand data retrieval.
- To produce room-wise student lists using advanced SQL string aggregation.

2.2 System Requirements

Component	Specification
Database	PostgreSQL 14+ / pgAdmin 4
Language	SQL, PL/pgSQL
OS	Windows 10/11, Ubuntu 20.04+
RAM	Minimum 4 GB
Storage	Minimum 500 MB free space
Tools	pgAdmin 4, VS Code (SQL extension), Terminal
Network	Local (standalone) or LAN

3. ENTITY-RELATIONSHIP DESCRIPTION

The Hostel Management System is built on four primary entities, each mapped to a relational table. The relationships between these entities are defined using primary and foreign keys, ensuring referential integrity and consistent data flow.

3.1 Entities and Their Attributes

Entity	Primary Key	Key Attributes	Relationship
Students	student_id	name, course, year	Has many allocations, many complaints
Rooms	room_id	capacity, occupied	Linked to many allocations
Allocation	allocation_id	student_id, room_id, allocation_date	Links Students ↔ Rooms
Complaints	complaint_id	student_id, issue, status	Belongs to one Student

3.2 Relationships Summary

Students to Allocation: One student can have one or more allocation records (one-to-many). Rooms to Allocation: One room can accommodate multiple students through multiple allocation records (one-to-many). Students to Complaints: One student can raise multiple complaints (one-to-many). The allocation table acts as a junction entity that resolves the many-to-many relationship between students and rooms.

3.3 Constraints Applied

Table	Constraint Type	Description
students	PRIMARY KEY	student_id uniquely identifies each student
rooms	PRIMARY KEY	room_id uniquely identifies each room
rooms	CHECK	occupied <= capacity (prevents overbooking)
allocation	FOREIGN KEY	student_id references students(student_id)
allocation	FOREIGN KEY	room_id references rooms(room_id)
complaints	FOREIGN KEY	student_id references students(student_id)
complaints	DEFAULT	status defaults to 'Pending' on insert

4. TABLE STRUCTURES & CREATION

4.1 Students Table

The students table stores essential information about each hostel resident including their name, academic course, and current year of study. The student_id is an auto-incrementing primary key.

```
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    name       VARCHAR(100),
    course     VARCHAR(50),
    year       INT
);
```

4.2 Rooms Table

The rooms table maintains a record of all hostel rooms, their total capacity, and the current number of students occupying them. The occupied field defaults to 0 upon creation of any new room record.

```
CREATE TABLE rooms (
    room_id  SERIAL PRIMARY KEY,
    capacity INT,
    occupied INT DEFAULT 0
);
```

4.3 Allocation Table

The allocation table records which student has been assigned to which room, along with the date of allocation. It references both the students and rooms tables via foreign keys, ensuring that only valid student and room IDs can be entered.

```
CREATE TABLE allocation (
    allocation_id  SERIAL PRIMARY KEY,
    student_id     INT REFERENCES students(student_id),
    room_id        INT REFERENCES rooms(room_id),
    allocation_date DATE DEFAULT CURRENT_DATE
);
```

4.4 Complaints Table

The complaints table captures all student-reported issues, the nature of the complaint, and the current resolution status. Status defaults to 'Pending' when a new complaint is filed and can be updated to 'In Progress' or 'Resolved' as the issue is handled.

```
CREATE TABLE complaints (
    complaint_id SERIAL PRIMARY KEY,
    student_id   INT REFERENCES students(student_id),
    issue        TEXT,
    status       VARCHAR(20) DEFAULT 'Pending');
```

5. CONSTRAINTS & TRIGGER

5.1 CHECK Constraint on Rooms

A CHECK constraint has been added to the rooms table to enforce a business rule: the number of students currently occupying a room (occupied) must never exceed its total capacity. This prevents overbooking and ensures data accuracy in room management.

```
ALTER TABLE rooms
ADD CONSTRAINT check_capacity
CHECK (occupied <= capacity);
```

5.2 Trigger Function — update_occupancy()

This PL/pgSQL function automatically increments the occupied count in the rooms table whenever a new allocation record is inserted. By using a trigger, the system removes the burden of manual updates from administrators and guarantees that occupancy counts are always accurate and up-to-date.

```
CREATE OR REPLACE FUNCTION update_occupancy()
RETURNS TRIGGER AS $$
BEGIN
   UPDATE rooms
   SET occupied = occupied + 1
   WHERE room_id = NEW.room_id;
   RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

5.3 Trigger — allocate_room_trigger

The trigger allocate_room_trigger fires AFTER each INSERT operation on the allocation table. It executes the update_occupancy() function on a per-row basis, ensuring that each new allocation immediately reflects in the rooms table's occupied count.

```
CREATE TRIGGER allocate_room_trigger
AFTER INSERT ON allocation
FOR EACH ROW
EXECUTE FUNCTION update_occupancy();
```

5.4 Function — get_student_room(sid INT)

This stored function accepts a student_id as a parameter and returns the corresponding room_id from the allocation table

```
CREATE OR REPLACE FUNCTION get_student_room(sid INT)
RETURNS TABLE(room_id INT) AS $$
BEGIN
   RETURN QUERY
   SELECT room_id FROM allocation WHERE student_id = sid;
END;
$$ LANGUAGE plpgsql;
```

6. DATA INSERTION (INSERT QUERIES)

6.1 Inserting Student Records

Sixteen student records are inserted into the students table covering multiple courses including BBA, BCA, B.Tech, MCA, MSC, MBA, and B.Com. Each student is identified by an auto-generated student_id.

```
INSERT INTO students (name, course, year) VALUES
('Aman Riyaz',        'BBA',    2),
('Riya Pal',          BCA',     1),
('Karan Kumar',       'B.Tech', 4),
('Priyanka Chandwani', 'MCA',   1),
('Sahil Hans',        'MCA',    2),
('Shubham Agarwal',   'MSC',    1),
('Harpreet Kaur',     'B.Tech', 1),
('Navdeep Singh',     'B.Tech', 2),
('Simran Sharma',     'BCA',    2),
('Rohit Verma',       'MBA',    1),
('Manpreet Gill',     'B.Tech', 3),
('Deepika Negi',      'MCA',    2),
('Arjun Malhotra',    'B.Com',  1),
('Jasleen Kaur',      'MBA',    2),
('Vishal Thakur',     'B.Tech', 4),
('Pooja Bhatia',      'MSC',    2);
```

Output: Students Inserted Successfully (Shown first 10 records)

	student_id [PK] integer	name character varying (100)	course character varying (50)	year integer
1	1	Aman Riyaz	BBA	2
2	2	Riya Pal	BCA	1
3	3	Karan Kumar	B.Tech	4
4	4	Priyanka Chandwani	MCA	1
5	5	Sahil Hans	MCA	2
6	6	Shubham Agarwal	MSC	1
7	7	Harpreet Kaur	B.Tech	1
8	8	Navdeep Singh	B.Tech	2
9	9	Simran Sharma	BCA	2
10	10	Rohit Verma	MBA	1

6.2 Inserting Room Records

Ten rooms are inserted with varying capacities ranging from single-occupancy rooms to four-person rooms. All rooms start with occupied = 0, which will be incremented automatically by the trigger as students are allocated.

```
INSERT INTO rooms (capacity, occupied) VALUES
(2, 0),  (3, 0),  (4, 0),  (2, 0),  (3, 0),
(4, 0),  (1, 0),  (2, 0),  (3, 0),  (4, 0);
```

Output: Rooms Inserted Successfully

	room_id [PK] integer	capacity integer	occupied integer
1	1	2	0
2	2	3	0
3	3	4	0
4	4	2	0
5	5	3	0
6	6	4	0
7	7	1	0
8	8	2	0
9	9	3	0
10	10	4	0

6.3 Inserting Allocation Records

Thirteen allocation records are inserted to assign students to specific rooms. The trigger fires automatically with each INSERT, updating the occupied count in the corresponding room. Note that room_id 7 (capacity 1) receives only one student as enforced by the CHECK constraint.

```
INSERT INTO allocation (student_id, room_id) VALUES
(1, 1),  (2, 3),  (3, 2),  (4, 7),  (5, 2),
(6, 5),  (7, 3),  (8, 6),  (9, 5),  (10, 8),
(11, 6), (12, 9), (13, 4);
```

Output: Allocations Inserted (Trigger fired for each row, Shown first 8 records)

	allocation_id [PK] integer	student_id integer	room_id integer	allocation_date date
1	1	1	1	2026-04-21
2	2	2	3	2026-04-21
3	3	3	2	2026-04-21
4	4	4	7	2026-04-21
5	5	5	2	2026-04-21
6	6	6	5	2026-04-21
7	7	7	3	2026-04-21
8	8	8	6	2026-04-21

6.4 Inserting Complaint Records

Ten complaint records are inserted covering various hostel issues. Complaints include infrastructure problems, maintenance requests, and quality concerns. Each complaint is linked to a valid student_id and defaults to 'Pending' status.

```
INSERT INTO complaints (student_id, issue, status) VALUES
(1,  'Water supply disrupted in bathroom since 2 days', 'Pending'),
(2,  'Wi-Fi connectivity very slow after 10 PM',        'Pending'),
(3,  'Room window latch broken, security concern',      'Resolved'),
(5,  'Mess food quality has degraded this week',        'Pending'),
(7,  'AC in room not functioning, unbearable',          'In Progress'),
(8,  'Cockroach infestation noticed near washroom',     'Pending'),
(10, 'Laundry machine on 2nd floor out of order',       'Resolved'),
(11, 'Common room TV remote is missing',                'Pending'),
(13, 'Power socket near study table not working',       'In Progress'),
(6,  'Noisy neighbour disturbing studies',              'Pending');
```

Output: Complaints Inserted Successfully

	complaint_id [PK] integer	student_id integer	issue text	status character varying (20)
1	1	1	Water supply disrupted in bathroom since 2 days	Pending
2	2	2	Wi-Fi connectivity very slow after 10 PM	Pending
3	3	3	Room window latch broken, security concern	Resolved
4	4	5	Mess food quality has degraded this week	Pending
5	5	7	AC in room not functioning, room temperature unbeara...	In Progress
6	6	8	Cockroach infestation noticed near washroom	Pending
7	7	10	Laundry machine on 2nd floor out of order	Resolved
8	8	11	Common room TV remote is missing	Pending
9	9	13	Power socket near study table not working	In Progress
10	10	6	Noisy neighbour disturbing studies late at night	Pending

7. SELECT QUERIES WITH OUTPUTS

Query 1: Students Allocated to Room 1

This query retrieves the names of all students who have been assigned to Room 1 by performing an INNER JOIN between the students and allocation tables, filtering by room_id = 1.

```
SELECT s.name
FROM students s
JOIN allocation a ON s.student_id = a.student_id
WHERE a.room_id = 1;
```

Output: Q1 — Students in Room 1

	name character varying (100)
1	Aman Riyaz

Query 2: Rooms with Available Space

This query fetches all rooms where the number of currently occupied beds is less than the total capacity, identifying rooms where new students can still be accommodated.

```
SELECT * FROM rooms WHERE occupied < capacity;
```

Output: Q2 — Available Rooms

	room_id [PK] integer	capacity integer	occupied integer
1	1	2	0
2	2	3	0
3	3	4	0
4	4	2	0
5	5	3	0
6	6	4	0
7	7	1	0
8	8	2	0
9	9	3	0
10	10	4	0

Query 3: All Pending Complaints

This query retrieves the complete details of all complaints that are currently in 'Pending' status — those that have been filed by students but not yet addressed by hostel administration.

```
SELECT * FROM complaints WHERE status = 'Pending';
```

Output: Q3 — Pending Complaints (6 records)

	complaint_id [PK] integer	student_id integer	issue text	status character varying (20)
1	1	1	Water supply disrupted in bathroom since 2 ...	Pending
2	2	2	Wi-Fi connectivity very slow after 10 PM	Pending
3	4	5	Mess food quality has degraded this week	Pending
4	6	8	Cockroach infestation noticed near washroo...	Pending
5	8	11	Common room TV remote is missing	Pending
6	10	6	Noisy neighbour disturbing studies late at ni...	Pending

Query 4: All Students Ordered by Course and Year

This query lists all students along with their course and year, sorted first alphabetically by course name and then by year within each course — useful for administrative course-wise grouping.

```
SELECT name, course, year
FROM students
ORDER BY course, year;
```

Output: Q4 — Students Ordered by Course & Year

	name character varying (100)	course character varying (50)	year integer
1	Arjun Malhotra	B.Com	1
2	Harpreet Kaur	B.Tech	1
3	Navdeep Singh	B.Tech	2
4	Manpreet Gill	B.Tech	3
5	Vishal Thakur	B.Tech	4
6	Karan Kumar	B.Tech	4
7	Aman Riyaz	BBA	2
8	Riya Pal	BCA	1
9	Simran Sharma	BCA	2
10	Rohit Verma	MBA	1
11	Jasleen Kaur	MBA	2
12	Priyanka Chandwani	MCA	1
13	Sahil Hans	MCA	2
14	Deepika Negi	MCA	2
15	Shubham Agarwal	MSC	1
16	Pooja Bhatia	MSC	2

Query 5: Final Year B.Tech Students

This query identifies B.Tech students who are in their 3rd or 4th year (final year phase), which may be needed for issuing clearances, priority room allocations, or alumni notifications.

```
SELECT name, course, year
FROM students
WHERE year > 3 AND course = 'B.Tech';
```

Output: Q5 — Final Year B.Tech Students

	name character varying (100)	course character varying (50)	year integer
1	Karan Kumar	B.Tech	4
2	Vishal Thakur	B.Tech	4

8. UPDATE QUERIES WITH OUTPUTS

Update Query 1: Mark Complaint #1 as Resolved

This targeted UPDATE sets the status of a specific complaint (complaint_id = 1) to 'Resolved'. This is used when a single complaint has been addressed and needs to be closed individually.

```
UPDATE complaints
SET status = 'Resolved'
WHERE complaint_id = 1;
```

Output: Complaint #1 status updated to 'Resolved'

	complaint_id [PK] integer	student_id integer	issue text	status character varying (20)
1	1	1	Water supply disrupted in bathroom since 2 d...	Resolved

Update Query 2: Bulk Resolve 'In Progress' Complaints

This bulk UPDATE marks all complaints that are currently 'In Progress' as 'Resolved'. This is efficient for end-of-day or periodic closure of resolved batches without updating records one by one.

```
UPDATE complaints
SET status = 'Resolved'
WHERE status = 'In Progress';
```

Output: 2 complaints moved from 'In Progress' to 'Resolved'

	complaint_id [PK] integer	student_id integer	issue text	status character varying (20)
1	3	3	Room window latch broken, security concern	Resolved
2	7	10	Laundry machine on 2nd floor out of order	Resolved
3	1	1	Water supply disrupted in bathroom since 2 days	Resolved
4	5	7	AC in room not functioning, room temperature unbeara...	Resolved
5	9	13	Power socket near study table not working	Resolved

9. DELETE & OTHER QUERIES WITH OUTPUTS

Delete Query 1: Remove a Resolved Complaint

This conditional DELETE removes complaint_id = 1 only if its status is 'Resolved'. The double condition ensures accidental deletion of active complaints is prevented — enforcing a safe-delete policy.

```
DELETE FROM complaints
WHERE complaint_id = 1 AND status = 'Resolved';
```

Output: Complaint #1 deleted successfully (1 row affected)

```
DELETE 1

Query returned successfully in 108 msec.
```

Advanced Query: Room-wise Student List using STRING_AGG

This advanced query combines three tables (rooms, allocation, students) using LEFT JOINs and uses STRING_AGG to concatenate all student names occupying the same room.

```
SELECT r.room_id, r.capacity, r.occupied,
       STRING_AGG(s.name, ', ') AS students_in_room
FROM rooms r
LEFT JOIN allocation a ON r.room_id = a.room_id
LEFT JOIN students s   ON a.student_id = s.student_id
GROUP BY r.room_id, r.capacity, r.occupied
ORDER BY r.room_id;
```

Output: Room-wise Consolidated Student List

	room_id [PK] integer	capacity integer	occupied integer	students_in_room text
1	1	2	0	Aman Riyaz
2	2	3	0	Karan Kumar, Sahil Hans
3	3	4	0	Harpreet Kaur, Riya Pal
4	4	2	0	Arjun Malhotra
5	5	3	0	Simran Sharma, Shubham Agar...
6	6	4	0	Navdeep Singh, Manpreet Gill
7	7	1	0	Priyanka Chandwani
8	8	2	0	Rohit Verma
9	9	3	0	Deepika Negi

10. CONCLUSION

The Hostel Management System successfully demonstrates the application of relational database concepts in a real-world administrative context. Through this project, we designed and implemented a complete PostgreSQL database that efficiently handles student data, room inventory, allocation records, and complaint tracking.

The use of foreign key constraints ensured referential integrity across all four tables, eliminating the risk of orphaned records or invalid data entries. The implementation of a CHECK constraint on the rooms table enforced business logic at the database level, preventing overbooking without requiring application-layer validation.

The AFTER INSERT trigger proved to be a powerful tool for automation — removing the manual burden of updating room occupancy after every allocation. The PL/pgSQL stored function get_student_room() illustrated how reusable logic can be encapsulated within the database, while the available_rooms view simplified real-time querying for hostel administrators.

The diverse set of SQL queries — ranging from simple SELECT filters to complex multi-table JOINs, GROUP BY aggregations, and STRING_AGG consolidation — demonstrates the expressive power and versatility of SQL in handling real administrative needs.

Key outcomes of this project include: 16 student records successfully managed, 10 rooms with dynamic occupancy tracking via triggers, 12 active allocations processed, 10 complaints tracked with multi-status updates, and over 20 SQL queries covering all CRUD operations demonstrated with outputs.

This project lays a solid foundation for a full-stack hostel management application that could be extended with a user interface built on frameworks like React or Django, REST API integration, role-based access control, and real-time notification systems for complaint resolution.

References

- PostgreSQL Documentation — Official PostgreSQL 14 Documentation. https://www.postgresql.org/docs/
- pgAdmin Documentation — https://www.pgadmin.org/docs/
- W3Schools SQL Tutorial — https://www.w3schools.com/sql/
- GeeksforGeeks — PostgreSQL Triggers and Functions — https://www.geeksforgeeks.org/postgresql-trigger/
- TutorialsPoint — SQL Constraints — https://www.tutorialspoint.com/sql/sql-constraints.htm